

Supervised Learning: Tree-based Learning and Ensemble Methods

Course Notes (COMP0078)

October 20, 2025

Revision notation: items marked with $\star$ are specifically listed in the revision notes file ([revision25-26-V1.pdf](#)). Non-examinable content (Boosting generalisation guarantees) has been removed.

1 Tree-based Learning Algorithms

1.1 Partitioning and Prediction

Tree methods partition the input space into a set of regions (hyper-rectangles) and fit a simple model (e.g., a constant) in each one.

Definition 1.1 (Tree Partition Model $\star$). *We partition the input space into hyper-rectangles $R_1, R_2, \dots, R_P$ where $\bigcup_{p=1}^P R_p = \mathbb{R}^n$ and $R_a \cap R_b = \emptyset$ if $a \neq b$. The prediction function is defined as:*

$$f(\mathbf{x}) = \sum_{p=1}^P c_p \mathcal{I}[\mathbf{x} \in R_p] \quad (1)$$

where $\mathcal{I}[\cdot]$ is the indicator function ($\mathcal{I}[\mathbf{x} \in R_p] = 1$ if $\mathbf{x} \in R_p$, 0 otherwise) and $\{c_p\}_{p=1}^P$ are real parameters.

A natural choice for the parameter c_p in Definition 1.1 is the average of the target values in that region:

$$c_p = \text{ave}(y_i | \mathbf{x}_i \in R_p) \equiv \frac{\sum_{i=1}^m y_i \mathcal{I}[\mathbf{x}_i \in R_p]}{\sum_{i=1}^m \mathcal{I}[\mathbf{x}_i \in R_p]} \quad (2)$$

1.2 Recursive Binary Partitions (CART) $\star$

1.2.1 Regression Trees

Definition 1.2 (Axis-Parallel Half-Planes $\star$). *We define a pair of axis-parallel half-planes based on a splitting variable j and split point s :*

$$R_1(j, s) = \{\mathbf{x} | x_j \leq s\}, \quad R_2(j, s) = \{\mathbf{x} | x_j > s\} \quad (3)$$

We search for the optimal $\bar{j}$ and $\bar{s}$ by solving the greedy splitting problem (using the half-planes of Definition 1.2):

$$\min_{j, s} \left\{ \min_{c_1} \sum_{\mathbf{x}_i \in R_1(j, s)} (y_i - c_1)^2 + \min_{c_2} \sum_{\mathbf{x}_i \in R_2(j, s)} (y_i - c_2)^2 \right\} \quad (4)$$

The inner minimization in (4) is solved by setting $\bar{c}_1, \bar{c}_2$ to the averages of y_i in the respective regions (as in (2)). The computational cost for a variable j is $O(m)$, leading to an overall complexity of $O(nm)$ to find the best split.

1.2.2 Pruning*

To avoid overfitting, we employ **Cost-Complexity Pruning**. We first grow a large tree $\hat{T}$ and then find a subtree $T_\lambda \subseteq \hat{T}$ minimizing the cost-complexity criterion:

Definition 1.3 (Cost-Complexity Criterion*).

$$C_\lambda(T) = \sum_{p=1}^{|T|} m_p Q_p(T) + \lambda |T| \quad (5)$$

where $|T|$ is the number of leaf nodes, m_p is the count of data points in node p , and

$$Q_p(T) = \frac{1}{m_p} \sum_{\mathbf{x}_i \in R_p} (y_i - c_p)^2 \quad (6)$$

is the mean squared error in node p .

Remark 1.4 (Regularisation Interpretation). The cost-complexity criterion (5) is a kind of regularised empirical error: $T_0 = \hat{T}$ (the original tree) and large values of λ result in smaller trees. A good value of λ can be obtained by cross-validation.

Weakest Link Pruning Algorithm:

1. Successively collapse the internal node that produces the smallest per-node increase in $\sum m_p Q_p(T)$ (as in Definition 1.3).
2. Continue until the single-node root tree is produced.
3. This produces a list of subtrees. One can show that T_λ is in this list, so the algorithm gives the optimal solution.
4. Select the optimal λ^* via cross-validation and retrain on all the data to find T_{λ^*} .

1.2.3 Classification Trees*

Definition 1.5 (Empirical Class Probabilities*). For categorical output $\mathcal{Y} = \{1, \dots, K\}$, we define the empirical class probabilities in each region R_p as:

$$\hat{p}_{pk} = \frac{1}{m_p} \sum_{\mathbf{x}_i \in R_p} \mathcal{I}[y_i = k] \quad (7)$$

We classify an input which falls in region p in the class with maximum probability:

$$f(\mathbf{x}) = \operatorname{argmax}_{k=1}^K \sum_{p=1}^N \hat{p}_{pk} \mathcal{I}[\mathbf{x} \in R_p] \quad (8)$$

Definition 1.6 (Node Impurity Measures*). Impurity measures $Q_p(T)$ used for classification trees include:

- **Misclassification error:** $1 - \hat{p}_{pk(p)}$, where $k(p) := \operatorname{argmax}_{k=1}^K \hat{p}_{pk}$
- **Gini index:** $\sum_k \hat{p}_{pk}(1 - \hat{p}_{pk})$
- **Cross entropy:** $\sum_k \hat{p}_{pk} \log \frac{1}{\hat{p}_{pk}}$

Gini and Cross-entropy are preferred for growing the tree due to higher sensitivity to node probability changes, while misclassification error is often used for pruning.

Example 1.7 (Gini vs. Entropy Splitting). *Consider splitting a node with $m_p = 800$ and two classes $\{A, B\}$ (400 each). Compare a left split (300A/100B, 100A/300B) versus a right split (200A/400B, 200A/0B):*

1. **Misclassification:** Left: $200 = (400 \times 1/4) + (400 \times 1/4)$; Right: $200 = (600 \times 1/3) + (200 \times 0)$. Both equal.
2. **Entropy:** Left: $649.02 = 400 \times (\frac{1}{4} \log 4 + \frac{3}{4} \log \frac{4}{3}) \times 2$; Right: $550.98 = 600 \times (\frac{1}{3} \log 3 + \frac{2}{3} \log \frac{3}{2}) + 200 \times 0$. The right split gives lower total entropy.

The right split is preferable since it produces a pure node, but misclassification error (Definition 1.6) fails to distinguish the two splits, whereas cross entropy does.

2 Ensemble Methods: Wisdom of the Crowd*

2.1 Analysis of Voting

Proposition 2.1 (Condorcet's Jury Theorem — Quantitative Form*). *Consider a crowd of $2T + 1$ voters $h_1, \dots, h_{2T+1}$. Suppose each voter predicts correctly with probability $\frac{1}{2} + \gamma$ independently. The majority vote is:*

$$H_T := \text{sign} \left(\sum_{t=1}^{2T+1} h_t \right) \quad (9)$$

The probability that H_T is wrong is given by:

$$P(H_T \text{ is wrong}) = \sum_{i=0}^T \binom{2T+1}{i} \left(\frac{1}{2} + \gamma \right)^i \left(\frac{1}{2} - \gamma \right)^{2T+1-i} \quad (10)$$

2.1.1 Chernoff Bound Derivation

Lemma 2.2 (Chernoff Bound). *Let $X_1, \dots, X_n$ be independent random variables with $0 \leq X_i \leq 1$. Let $\mu = E[\sum_{i=1}^n X_i]$. Then for $0 \leq k \leq \mu$:*

$$P \left(\sum_{i=1}^n X_i \leq k \right) \leq \exp \left(-\frac{(\mu - k)^2}{2\mu} \right) \quad (11)$$

Corollary 2.3 (Exponential Decay of Crowd Error). *Under the setting of Proposition 2.1, applying the Chernoff bound (Lemma 2.2) with $X_i = 1$ if voter i is correct, $n = 2T + 1$, $k = T$, and $\mu = (2T + 1)(\frac{1}{2} + \gamma) = T + \frac{1}{2} + 2T\gamma + \gamma$:*

$$P(H_T \text{ is wrong}) \leq \exp \left(-\frac{(\mu - T)^2}{2\mu} \right) \quad (12)$$

$$\begin{aligned} &= \exp \left(-\frac{(\frac{1}{2} + 2T\gamma + \gamma)^2}{2(T + \frac{1}{2} + 2T\gamma + \gamma)} \right) \\ &\leq \exp \left(-\frac{4T^2\gamma^2}{5T} \right) \quad (\text{simplifying for large } T) \\ &\leq \exp \left(-\frac{4\gamma^2}{5} T \right) \end{aligned} \quad (13)$$

The probability that the crowd is wrong decays exponentially to zero as T increases.

3 Bagging and Random Forests*

3.1 Bagging Algorithm

Definition 3.1 (Bagging*). *Idea: Reduce variance by training classifiers on bootstrap samples and voting.*

1. *Input: Data $\mathcal{S} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\} \subset \mathbb{R}^d \times \{-1, 1\}$, ensemble size T , resample size M (usually $M = m$), classifier function $h_{\mathcal{S}}(\mathbf{x})$.*
2. *For $t = 1$ to T : Sample $S^{(t)}$ of size M with replacement from $\mathcal{S}$. Train $h_{S^{(t)}}$.*
3. *Return:*

$$H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T h_{S^{(t)}}(\mathbf{x}) \right) \quad (14)$$

Remark 3.2 (Bootstrap Coverage). *The probability a specific example is not selected in one bootstrap sample is $(1 - 1/m)^m \rightarrow 1/e \approx 0.368$ as $m \rightarrow \infty$. Thus, each bag contains roughly 63% distinct examples.*

3.2 Random Forests

Definition 3.3 (Random Forest*). *To make the “wisdom of crowds” argument (Proposition 2.1) work, predictors must be independent. Bagged trees (Definition 3.1) are correlated because they all consider the same features at each split.*

Random Forest Idea: *De-correlate trees by checking only a random subset of size k features at each split (typically $k = \sqrt{n}$ or $\log n$).*

4 Boosting*

4.1 Concepts and Algorithm

Definition 4.1 (Weak Learner*). *A **weak learner** is an algorithm which can consistently find classifiers (“rules of thumb”) at least slightly better than random guessing, i.e., with accuracy $> 50\%$.*

Definition 4.2 (Boosting*). ***Boosting** is a general method of converting rough rules of thumb (weak learners, Definition 4.1) into a highly accurate prediction rule (classifier).*

Definition 4.3 (AdaBoost Glossary*). *We define the following quantities used in AdaBoost:*

- $D_t(i)$: *the weight on example i at time t , where $\sum_{i=1}^m D_t(i) = 1$.*
- $\alpha_t \in \mathbb{R}$: *the weight on weak learner t .*
- $h_t(\cdot) : \mathbb{R}^d \rightarrow \{-1, 1\}$: *the weak learner generated at time t .*
- $f(\mathbf{x}) := \sum_{t=1}^T \alpha_t h_t(\mathbf{x})$: *the aggregate scoring function.*
- $H(\mathbf{x}) := \text{sign}(f(\mathbf{x}))$: *the final classifier output.*
- ϵ_t : *the weighted error of weak learner h_t at time t :*

$$\epsilon_t := \sum_{i=1}^m D_t(i) \mathcal{I}[h_t(\mathbf{x}_i) \neq y_i] \quad (15)$$

Algorithm 1 AdaBoost*

Require: Training data $S = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$, number of rounds T

- 1: Initialise $D_1(i) = 1/m$ for all $i \in \{1, \dots, m\}$
- 2: **for** $t = 1, \dots, T$ **do**
- 3: Fit classifier $h_t : \mathbb{R}^d \rightarrow \{-1, 1\}$ using distribution D_t
- 4: Compute weighted error ϵ_t as in (15)
- 5: Set $\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$ (see Lemma 4.6)
- 6: Update: for each $i \in \{1, \dots, m\}$:

$$D_{t+1}(i) = \frac{D_t(i) \exp(-\alpha_t y_i h_t(\mathbf{x}_i))}{Z_t} \quad (16)$$

where Z_t is a normalisation factor ensuring $\sum_{i=1}^m D_{t+1}(i) = 1$.

7: **end for**

8: **return** Final classifier $H(\mathbf{x}) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x})\right)$

Remark 4.4 (AdaBoost Weight Dynamics). *At each round of Algorithm 1, larger weights are assigned to hard examples (those previously misclassified), so the weak learner will focus on those examples. The final classifier is a weighted majority vote of the T base classifiers, where α_t is the weight assigned to h_t . Since $\epsilon_t \leq 0.5$ implies $\alpha_t \geq 0$, the function f is a non-negative linear combination of the h_t with weights controlled by the training error.*

4.2 Convergence Analysis (Detailed Proof)

Theorem 4.5 (AdaBoost Training Error Bound*). *Given a training set $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)\}$ and assume that at each iteration of AdaBoost (Algorithm 1) the weak learner returns a hypothesis with weighted error $\epsilon_t \leq \frac{1}{2} - \gamma$. Then the training error of the output hypothesis satisfies:*

$$\frac{1}{m} \sum_{i=1}^m \mathcal{I}[H(\mathbf{x}_i) \neq y_i] \leq \exp(-2\gamma^2 T) \quad (17)$$

Proof of Theorem 4.5. Let $f(\mathbf{x}) = \sum_{t=1}^T \alpha_t h_t(\mathbf{x})$ (as in Definition 4.3).

Step 1 (Exponential upper bound on 0-1 loss). Since $H(\mathbf{x}_i) \neq y_i \implies y_i f(\mathbf{x}_i) \leq 0 \implies \exp(-y_i f(\mathbf{x}_i)) \geq 1$:

$$\frac{1}{m} \sum_{i=1}^m \mathcal{I}[H(\mathbf{x}_i) \neq y_i] \leq \frac{1}{m} \sum_{i=1}^m e^{-y_i f(\mathbf{x}_i)} \quad (18)$$

Step 2 (Unrolling the recursion). From the update rule (16):

$$D_{T+1}(i) = \frac{e^{-y_i f(\mathbf{x}_i)}}{m \prod_{t=1}^T Z_t} \quad (19)$$

Hence, using that $\sum_{i=1}^m D_{T+1}(i) = 1$:

$$\frac{1}{m} \sum_{i=1}^m e^{-y_i f(\mathbf{x}_i)} = \sum_{i=1}^m D_{T+1}(i) \prod_{t=1}^T Z_t = \prod_{t=1}^T Z_t \quad (20)$$

Step 3 (Evaluating Z_t). Since h_t maps to $\{-1, 1\}$, using the update formula (16):

$$Z_t = \sum_i D_t(i) e^{-\alpha_t y_i h_t(\mathbf{x}_i)} = \epsilon_t e^{\alpha_t} + (1 - \epsilon_t) e^{-\alpha_t} \quad (21)$$

Setting $\frac{dZ_t}{d\alpha_t} = 0$ yields $\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$ (see Lemma 4.6).

Step 4 (Substituting optimal α_t). Substituting into (21):

$$Z_t = 2\sqrt{\epsilon_t(1-\epsilon_t)} = \sqrt{1-4\gamma_t^2} \quad (22)$$

where $\gamma_t = 1/2 - \epsilon_t$.

Step 5 (Applying $1-x \leq e^{-x}$). Combining (18), (20), and (22):

$$\prod_{t=1}^T Z_t = \prod_{t=1}^T \sqrt{1-4\gamma_t^2} \leq \prod_{t=1}^T e^{-2\gamma_t^2} = e^{-2\sum_{t=1}^T \gamma_t^2} \quad (23)$$

If $\gamma_t \geq \gamma$ for all t , the training error $\leq e^{-2T\gamma^2}$. $\square$

Lemma 4.6 (Optimal AdaBoost Weight*). *The normalisation factor Z_t in (16) is minimised by the choice:*

$$\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t} \quad (24)$$

Proof. From (21), $Z_t = \epsilon_t e^{\alpha_t} + (1-\epsilon_t)e^{-\alpha_t}$. Setting $\frac{dZ_t}{d\alpha_t} = \epsilon_t e^{\alpha_t} - (1-\epsilon_t)e^{-\alpha_t} = 0$ gives $e^{2\alpha_t} = \frac{1-\epsilon_t}{\epsilon_t}$, hence $\alpha_t = \frac{1}{2} \log \frac{1-\epsilon_t}{\epsilon_t}$. $\square$

4.3 Additive Models and Loss Functions*

Definition 4.7 (Forward Stagewise Additive Model*). *Boosting can be seen as a greedy solution to:*

$$\min \left\{ \sum_{i=1}^m V \left(y_i, \sum_{t=1}^T \alpha_t h_t(\mathbf{x}_i) \right) : \alpha_1, \dots, \alpha_T \in \mathbb{R}^T, h_1, \dots, h_T \in \mathcal{H}^T \right\} \quad (25)$$

where $\mathcal{H}$ is the hypothesis class containing the weak learners (Definition 4.1) and V is the exponential loss $V(y, \hat{y}) = \exp(-y\hat{y})$. At each iteration a new basis function is added to the current expansion $f^{(t-1)} = \sum_{s=1}^{t-1} \alpha_s h_s$.

$$(\alpha_t, h_t) = \operatorname{argmin}_{\alpha_t, h_t} \sum_{i=1}^m V(y_i, f^{(t-1)}(\mathbf{x}_i) + \alpha_t h_t(\mathbf{x}_i)) \quad (26)$$

Proposition 4.8 (Derivation of AdaBoost from Exponential Loss*). *Substituting $V(y, \hat{y}) = \exp(-y\hat{y})$ into the stagewise objective (26) and defining $D_t(i) := e^{-y_i f^{(t-1)}(\mathbf{x}_i)}$, we obtain:*

$$\min_{\alpha_t, h_t} \sum_{i=1}^m D_t(i) e^{-\alpha_t y_i h_t(\mathbf{x}_i)} \quad (27)$$

Since h_t maps to $\{-1, 1\}$, this can be rewritten as:

$$\min_{\alpha_t, h_t} \left\{ (e^{\alpha_t} - e^{-\alpha_t}) \sum_{i=1}^m D_t(i) \mathcal{I}[y_i \neq h_t(\mathbf{x}_i)] + e^{-\alpha_t} \sum_{i=1}^m D_t(i) \right\}$$

This shows that:

1. The h_t minimising the above minimises the misclassification error weighted by D_t (recovering the weak learner selection of Algorithm 1).
2. The D_t defined here is proportional to the AdaBoost distribution (see (19)).
3. The minimisation over α_t recovers the optimal weight (Lemma 4.6).

4.3.1 Why Exponential Loss?

Definition 4.9 (Classification Loss Functions*). Consider the following typical loss functions for binary classification with margin $y\hat{y}$, where $y \in \{-1, 1\}$:

$$V_{mc}(y, \hat{y}) = \mathcal{I}[y \neq \text{sign}(\hat{y})] \quad (\text{misclassification}) \quad (28)$$

$$V_{\text{hinge}}(y, \hat{y}) = \max(0, 1 - y\hat{y}) \quad (\text{hinge}) \quad (29)$$

$$V_{\text{exp}}(y, \hat{y}) = \exp(-y\hat{y}) \quad (\text{exponential}) \quad (30)$$

$$V_{sq}(y, \hat{y}) = (y - \hat{y})^2 \quad (\text{square}) \quad (31)$$

Remark 4.10 (Loss Function Comparison*). Comparing the loss functions of Definition 4.9:

1. Misclassification (28) is not continuous, hence difficult to optimise.
2. Square loss (31) unnecessarily punishes predictions with increasing positive margin (correct predictions that are “too confident”).
3. Hinge loss (29) has the nice property of penalising negative margins but not positive margins; however, it is not differentiable everywhere.
4. Exponential loss (30) is differentiable, punishes negative margins heavily, and promotes large positive margins (secondarily). This makes it suitable for the forward stagewise procedure (Definition 4.7).

5 Problems

Problem 1 (Regression Trees). Given the dataset $\{((1, 1), 9), ((1, 2), -4), ((1, 3), 2), ((2, 2), 4), ((2, 3), 2)\}$:

1. Find the regression tree corresponding to the greedy recursive partitioning procedure (as in (4)) with respect to square error.
2. Draw the recursive partition.
3. Draw the tree representation.

Problem 2 (Random Forests). Explain how a random forest (Definition 3.3) is constructed. What is the motivation for the tree construction method?

Problem 3 (Ensemble Structures). Both bagging (Definition 3.1) and boosting (Algorithm 1) produce predictions by creating an ensemble of functions. Explain how these ensembles differ structurally (not the method of arrival, but the structure of the final ensemble).

Problem 4 (AdaBoost Initialization and Update). 1. Explain how AdaBoost (Algorithm 1) chooses its initial weighting $D_1(i)$.

2. Explain how it updates the distribution D_t after choosing weak learner h_t with coefficient α_t , including the normalizing constant Z_t (see (16)).
3. Give an intuitive justification for this update.

Problem 5 (AdaBoost Distribution Usage). Explain how AdaBoost uses the distribution D_t (Definition 4.3) to choose a weak learner from a set $\mathcal{H}$ at stage t .

Problem 6 (Bounding the Indicator). Using the result (20), obtain a bound for:

$$\frac{1}{m} \sum_{i=1}^m \mathcal{I}[\text{sign}(f_T(\mathbf{x}_i)) \neq y_i] \leq \frac{1}{m} \sum_{i=1}^m \exp(-y_i f_T(\mathbf{x}_i))$$

in terms of the normalization constants $Z_1, \dots, Z_T$.

Problem 7 (Weakly-Universal Classes). 1. A dataset $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_m, y_m)$ is generic if $\mathbf{x}_i = \mathbf{x}_j \implies y_i = y_j$. A set of functions $\mathcal{H}$ is weakly-universal if for every generic dataset and weighting, there exists $h \in \mathcal{H}$ with weighted error < 0.5 .

2. Is the set of decision stumps $\mathcal{H}_{ds}$ weakly-universal? Provide an argument.

3. Let S be the set of finite non-zero strings over $\{a, b\}$. Define substring matching functions $g_z(x) = 1$ if $x \sqsubseteq z$, else -1 . Is the set $\mathcal{H}_{ss}$ of substring-match functions weakly-universal? Provide an argument.

Shuhan Wang